

COSC264 · INTRODUCTION TO COMPUTER NETWORKS AND THE INTERNET · TERM 4

Module 3

Error Detection and Correction

cosc264.up.railway.app

A quick recap of last week

There are two kinds of routing: **intra-AS routing**, inside a single autonomous system, and **inter-AS routing**, between them.

For **intra-AS routing** we looked at **RIP**, a distance-vector protocol, and **OSPF**, a link-state one.

For **inter-AS routing** there is effectively only one answer, **BGP**: **eBGP** runs between ASes and learns the AS-level path to prefixes in other ASes, and **iBGP** carries what was learned to every BGP speaker inside the AS.

We also looked at **scaling the IPv4 address space**, by two main mechanisms: **NAT** and **IPv6**.

MODULE 3 · CONCEPT 1 OF 3

Error Detection

1 · Error Detection

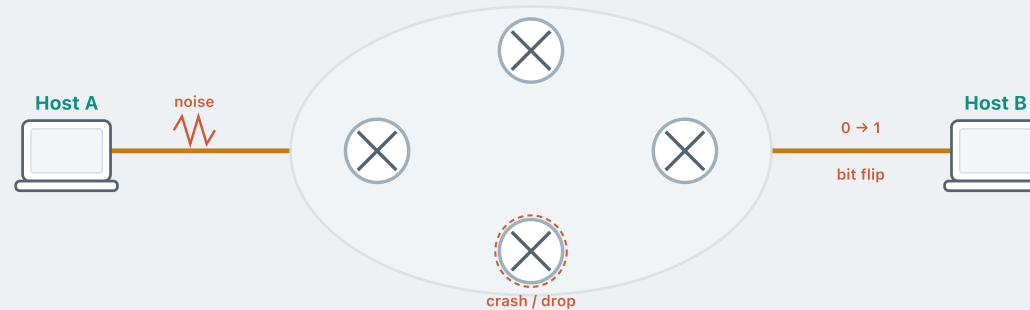
2 · CRC

3 · Error Correction

MOTIVATION

The journey of a packet

Consider sending a datagram from **host A** to **host B** across a network of links and routers.



Along the way — **thermal noise**, weak signals, interference, equipment crashes — the packet might arrive **corrupted**, arrive **out of order**, or be **dropped** entirely.

Three categories of problem

PROBLEM	APPROACH
Corrupted packets	Error detection and correction this module
DEFINITION A packet is corrupted when it is delivered, but one or more of its bits have been altered in transit. Those altered bits are called bit errors.	
Lost packets	ARQ protocols Module 4
Out-of-order packets	ARQ protocols Module 4

Types of bit errors



Burst error of length B — the span from the first to the last corrupted bit is B positions. The bits in between **may or may not be flipped**: one disturbance is continuous in time, but not every bit it touches comes out wrong.

Single-bit error — a single isolated bit is altered.

The error detection process

The sender runs the data through an **error-detecting code**, which produces a small number of **check bits** E — a compact fingerprint that depends on every bit of the original content.

TRANSMITTER



The error detection process

RECEIVER

1. Receive and parse: $\boxed{\text{Data}'}$ $\boxed{\text{E}}$ $\rightarrow$ $\boxed{\text{Data}'}$ k bits $+$ $\boxed{\text{E}}$ check bits (r)
2. $\boxed{\text{Data}'}$ $\rightarrow$ $\boxed{\text{recompute check bits}}$ $\rightarrow$ $\boxed{\text{E}'}$ check bits (r)
3. Compare E (received) with E' (recomputed)

$\text{E} = \text{E}' \rightarrow$ no error detected — accept the data.

$\text{E} \neq \text{E}' \rightarrow$ error detected — discard or request retransmission.

- If nothing is corrupted, the check **always** passes.
- The reverse is **not** true. A check that passes does not guarantee nothing was corrupted — it only says **no error was detected**. Some rare corruption patterns still slip through.
- So a **good** error-detecting code should catch **as many error patterns as it can**: if an error occurred, it is detected with **high probability**.

Parity check

The simplest **error-detecting** scheme: append a **parity bit** to each block of data.

Even parity — parity bit makes the total number of 1s (including the parity bit) even.

TRANSMITTER

1.

1 0 0 0 0 1 0 1
data (3 ones)

 →

even parity

 →

1
parity bit

 $3 + 1 = 4$ (even) ✓

2. Append:

1 0 0 0 0 1 0 1

1

 → transmit

PARITY CHECK

Parity check — detecting the error

IN TRANSIT

Sent: 1 0 0 0 0 1 0 1 1 bit 4 flips → Received: 1 0 0 1 0 1 0 1 1 — the parity bit itself arrives unchanged

RECEIVER

1. Parse: 1 0 0 1 0 1 0 1 data' (4 ones) + 1 received
2. 1 0 0 1 0 1 0 1 → even parity → 0 recomputed
3. Compare: **1** (received) ≠ **0** (recomputed) → **Error detected!**

Odd parity is the same idea with the opposite convention — the parity bit makes the total number of 1s **odd**.

Parity check — can it be fooled?

Can you think of a scenario where the parity check **fails** — the data arrives corrupted, but the receiver sees nothing wrong?

Take the same even-parity example, and let **two** bits flip — one of them the parity bit itself.

IN TRANSIT — 2 BITS FLIPPED

1 0 0 0 0 1 0 1 | 1 → 1 0 0 1 0 1 0 1 | 0 bit 4: 0→1, parity bit: 1→0

RECEIVER

1. Parse:

1 0 0 1 0 1 0 1
data' (4 ones)

 +

0
received
2.

1 0 0 1 0 1 0 1

 →

even parity

 →

0
recomputed
3. Compare: **0 = 0** → **No error detected!**

The limitation of a parity check

A parity check cannot detect a corruption pattern in which an **even number of bits** are flipped.

The reason is simple: adding an **even number** to a count never changes whether that count is odd or even. The recomputed parity bit comes out the same as the one received, and the check passes.

So we need a more **robust** error-detection scheme.

How? Intuitively, by spending **more check bits**.

That is the idea behind the **two-dimensional parity check**.

2-dimensional parity check

Arrange the data into a **matrix** of rows and columns.

For example, a 20-bit data block becomes the 4×5 matrix on the right.

Append a **row parity bit** r_i to each row (even parity).

Append a **column parity bit** c_j to each column.

An **overall parity bit** p completes the matrix.

0	1	1	1	0	1
0	1	1	1	0	1
0	1	0	0	0	1
0	1	0	1	1	1
0	0	0	1	1	0

↑ column parity row parity →

2-D PARITY

How much can 2-D parity catch?

Flip **two bits in the same row** — the pattern that defeated the one-dimensional check.

Can this error be detected by the 2-D parity check?

The **row** check still passes: within that row, one flip cancels the other.

But each flip is alone in its **column**, so it flips that column's parity — the **column check catches the error**.

In general, every pattern of **one, two or three** flipped bits is detected.

Can you think of an error pattern that 2-D parity would miss?

0	1	1	1	0	1	✓
0	0	0	1	0	1	✓
0	1	0	0	0	1	✓
0	1	0	1	1	1	✓
0	0	0	1	1	0	
✓	x	x	✓	✓		

How much can 2-D parity catch?

Flip **two bits in the same row** — the pattern that defeated the one-dimensional check.

Can this error be detected by the 2-D parity check?

The **row** check still passes: within that row, one flip cancels the other.

But each flip is alone in its **column**, so it flips that column's parity — the **column check catches the error**.

In general, every pattern of **one, two or three** flipped bits is detected.

Can you think of an error pattern that 2-D parity would miss?

Consider the four flipped bits on the right, at the corners of a **rectangle** — every check **passes**, and the error goes **undetected**.

0	0	0	1	0	1	✓
0	0	0	1	0	1	✓
0	1	0	0	0	1	✓
0	1	0	1	1	1	✓
0	0	0	1	1	0	
✓	✓	✓	✓	✓		

2-D PARITY

Correcting a single-bit error

2-D parity can not only **detect** an error — if just a single bit was flipped, it can also **correct** it.

The matrix on the right is as transmitted.

During transmission, a single bit is **flipped**.

The receiver's parity check **fails for row 2** and **column 2**.

The failing row and column **intersect** at the corrupted bit — flip it to **correct** the error.

0	1	1	1	0	1	✓
0	1	1	1	0	1	✗
0	1	0	0	0	1	✓
0	1	0	1	1	1	✓
0	0	0	1	1	0	
✓	✗	✓	✓	✓		

Internet checksum

Parity is the simplest form of the idea. The **Internet checksum** is the scheme real protocols actually carry.

The **Internet checksum** is a specific **error-detecting code** used in **IPv4**, **TCP**, and **UDP** (discussed later) to let the receiver detect bit errors introduced during transmission.

IPv4 HEADER — 32 bits

Version	IHL	Diff. Services	Total Length	
Identification			Flags	Fragment Offset
TTL	Protocol		Header Checksum	
Source Address				
Destination Address				
Options (0 or more words)				

TCP HEADER — 32 bits

Source Port			Destination Port		
Sequence Number					
Acknowledgement Number					
Data Offset	Rsv	Flags		Window Size	
Checksum				Urgent Pointer	
Options (0 or more 32-bit words)					
Data (optional)					

One's complement arithmetic

The checksum uses two operations:

One's complement addition

1. Add the two numbers as unsigned integers.
2. If there is a carry out of the leftmost bit, add 1 to the sum (*end-around carry*).

ex1. (no carry)

```
A2B3
+ 1D4C
-----
BFFF
```

ex2. (carry out)

```
A2B3
+ 6F8E
-----
carry → 11241
+    1 ← end-around
-----
1242
```

One's complement operation

Replace every 0 bit with 1 and every 1 bit with 0.

1111 → ones complement → 0000

1001 → ones complement → 0110

Adding a value and its one's complement always gives all 1s:

1001 + 0110 = 1111

Sender computation

- 1. Take the **IP header**.
- 2. Split it into **16-bit words**.
- 3. Compute their **one's-complement sum**.
- 4. Take the **one's complement** of that sum — that is the **checksum**.

IPv4 header rows are 32 bits, so each can be written as **8 hex digits**:

F2345678

F234 5678 32 bits → two 16-bit words

	HEX	BINARY
Word 1	F234	1111 0010 0011 0100
Word 2	5678	0101 0110 0111 1000
Add	148AC	1 0100 1000 1010 1100
Wrap carry	+ 1	+ 1
One's-complement sum	48AD	0100 1000 1010 1101
Checksum = complement	B752	1011 0111 0101 0010

Receiver verification

- 1. Take the received header — the checksum field now holds **B752**.
- 2. Compute the **one's-complement sum** of the header words.
- 3. Add the **checksum** to that sum.
- 4. The checksum is the **one's complement** of that sum, so adding them must give **all 1 bits** — check that the total is **FFFF**.

F234 5678 **B752**

	HEX	BINARY
Word 1	F234	1111 0010 0011 0100
Word 2	5678	0101 0110 0111 1000
Add words 1–2	148AC	1 0100 1000 1010 1100
Wrap carry	+ 1	+ 1
One's-complement sum	48AD	0100 1000 1010 1101
Add checksum	+ B752	+ 1011 0111 0101 0010
Total	FFFF	1111 1111 1111 1111

Efficiency and limitations

The primary reason for its adoption is **efficiency**:

- Most of these protocols are implemented in software
- The Internet checksum involves simple operations → very little computational overhead

But it also has a **limitation**:

- It is much less effective at detecting errors than the **cyclic redundancy check (CRC)**, which we look at next

MODULE 3 · CONCEPT 2 OF 3

Cyclic Redundancy Check

1 · Error Detection

2 · **CRC**

3 · Error Correction

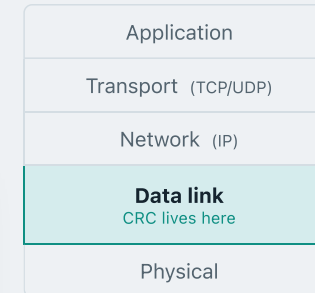
Cyclic Redundancy Check (CRC)

The CRC is one of the **most common and most powerful** error-detecting codes in use.

It lives in the **data link layer**. Every frame you send over Ethernet or Wi-Fi carries a CRC.

The intuition

- This time we read the message as a **number** — one big integer, rather than a string of bits.
- Sender and receiver agree on a fixed **divisor** in advance.
- The sender appends a few **check bits**, chosen so that the whole thing — read as one number — is **exactly divisible** by that divisor.



CRC — notation

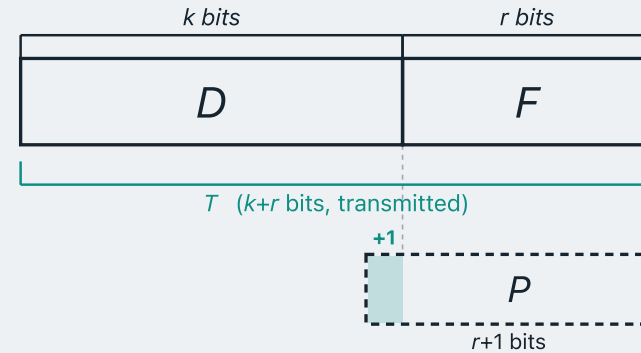
D — the k -bit block of data to send

P — the **divisor**, a fixed pattern of $r+1$ bits that sender and receiver agree on in advance

F — the **frame check sequence (FCS)** we must compute, r bits long

T — the frame actually transmitted, $k+r$ bits: D followed by F

Some texts write F 's length as $n-k$, where $n = k+r$ is the whole frame.



How to choose the pattern P?

Not every pattern will do — three **clearly bad** choices:

- **P = 0000...0** — you cannot divide by zero at all.
- **Highest-order bit of P = 0** — the leading zero contributes nothing to the division, so P is effectively **one bit shorter**. And since the FCS is always one bit shorter than P, the **effective FCS is one bit shorter** too — fewer check bits than we intended.
- **Lowest-order bit of P = 0** — the trailing zero contributes nothing either, unless the burst happens to reach the very last bit of the frame. Anywhere else, P again acts **one bit shorter**.

A zero at either end wastes a check bit — and fewer check bits means weaker error detection.

The exact bit pattern depends on the **type of errors** expected — but in every case:

REQUIRED

The **highest-order bit** = 1

REQUIRED

The **lowest-order bit** = 1

Together these two buy the **burst-error guarantee** — we will see why shortly.

Modulo-2 Arithmetic

- Addition and subtraction are both performed with **XOR ($\oplus$)** — so they are the **same operation**.

ADDITION (XOR)

```
  1111
⊕ 1010
-----
  0101
```

SUBTRACTION (XOR)

```
  1111
⊕ 1010
-----
  0101
```

- Multiplication is the usual shift-and-add — only the additions inside it become XORs.

MULTIPLICATION

```
  11001
×   11
-----
  11001
⊕ 110010
-----
 101011
```

How to compute the FCS

GIVEN

D **1010001101** $k = 10$ bits

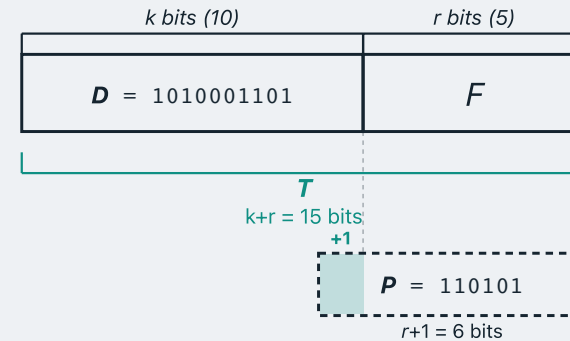
P **110101** $r+1 = 6$ bits

TO FIND

F , the **FCS** — one bit shorter than P , so $r = 5$ bits — such that:

1. appending F to D gives T , of $k+r = 10 + 5 = 15$ bits;
2. as integers, $T = D \times 2^r + F$;
3. and T is **exactly divisible by P** .

$$F = \text{remainder of } (D \times 2^r) / P$$



Plugging in the numbers:

$$\begin{aligned} F &= \text{rem. of } (1010001101 \times 2^5) / 110101 \\ &= \text{rem. of } 101000110100000 / 110101 \end{aligned}$$

An Example — Computing FCS

```

          1101010110    quotient (discarded)
          -----
110101 ) 101000110100000    D × 25
          110101000000000    ⊕ P
          -----
          11101110100000
          110101000000000    ⊕ P shifted
          -----
          111010100000
          : four more steps :
          -----
          01110    remainder
  
```

We need the remainder of $101000110100000 \div 110101$, and we get it by **long division** — the same procedure you learned at school.

The only difference: at each step the subtraction is an **XOR**, because we are working in **modulo-2 arithmetic**.

$F = 01110$ (the 5-bit FCS)

Transmitted frame T :

1010001101**01110**

The receiver's check

```

          1101010110 quotient (discarded)
          -----
110101 ) 101000110101110 received T
        1101010000000000 ⊕ P
        -----
          11101110101110
          1101010000000000 ⊕ P shifted
          -----
            111010101110
              : four more steps :
              -----
                00000 remainder
  
```

The receiver gets the transmitted frame T and knows the divisor P . To check whether the frame is corrupted, it simply checks whether T is **divisible by P** — again, by long division.

Remainder = 0

In this case T is divisible by P , so the receiver declares that **no error is detected**.

One more way to write the same thing

The divisor P is also called the **generator** — it is what generates the FCS.

A bit string can also be written as a **polynomial**, and generators are almost always written that way.

Two reasons to use the polynomial representation:

- Every CRC **standard** publishes its generator in this form.
- It is the form in which you can **analyse** which errors a generator will catch.

Bit strings as polynomials

Each bit becomes one term: the **power** of x gives the bit's **position**, the **coefficient** gives its **value**. Only the 1-bits survive.

x^9	x^8	x^7	x^6	x^5	x^4	x^3	x^2	x^1	x^0
1	0	1	0	0	0	1	1	0	1

→

$x^9 + x^7 + x^3 + x^2 + 1$

binary → polynomial

Going back, write a 1 wherever a power appears and a 0 wherever it does not — here x^6 , x^3 , x^2 and x^1 are all missing.

$x^7 + x^5 + x^4 + 1$

→

x^7	x^6	x^5	x^4	x^3	x^2	x^1	x^0
1	0	1	1	0	0	0	1

polynomial → binary

The same FCS, two representations

BIT STRINGS

D 1010001101

P 110101

F = remainder of
101000110100000 $\div$ 110101

POLYNOMIALS

$D(x) = x^9 + x^7 + x^3 + x^2 + 1$

$P(x) = x^5 + x^4 + x^2 + 1$

$F(x)$ = remainder of
?

How do we write "append five zeros to D , then divide by P " in polynomial form?

The same FCS, two representations

BIT STRINGS

D 1010001101

P 110101

F = remainder of
101000110100000 $\div$ 110101

POLYNOMIALS

$D(x) = x^9 + x^7 + x^3 + x^2 + 1$

$P(x) = x^5 + x^4 + x^2 + 1$

$F(x)$ = remainder of
 $D(x) \cdot x^5 \div P(x)$

How do we write “append five zeros to D , then divide by P ” in polynomial form?

Multiply $D(x)$ by x^5 and divide by $P(x)$ — it does exactly the same thing. Worth checking for yourself.

You do not need to learn polynomial division. If it is unfamiliar, work in bit strings and convert the remainder at the end.

F 01110

$F(x) = x^3 + x^2 + x$

T 101000110101110

$T(x) = x^{14} + x^{12} + x^8 + x^7 + x^5 + x^3 + x^2 + x$

Why is this representation useful?

Optional material, for anyone curious about *why* the polynomial form is used. Not examined.

It all comes down to one question: **how do we choose a good generator $P(x)$?**

OBSERVATION

A corrupted frame can always be written as $T(x) + E(x)$, where the **error polynomial** $E(x)$ has a 1 in every flipped position.

CRC builds $T(x)$ so that $P(x)$ divides it **exactly**. So the receiver's check is simply: is $T(x) + E(x)$ divisible by $P(x)$?

Therefore the error slips through **exactly when $P(x)$ divides $E(x)$** .

So a well-chosen $P(x)$ is one that **cannot divide** the error patterns we expect to see — which is exactly what makes it catch most of them.

Example: a short burst error

Our generator is $P(x) = x^5 + x^4 + x^2 + 1$, of **degree 5**.

The **degree** of a polynomial is its highest power — equivalently, the position of its leading 1 bit.

	x^{14}	x^{13}	x^{12}	x^{11}	x^{10}	x^9	x^8	x^7	x^6	x^5	x^4	x^3	x^2	x^1	x^0
Sent T	1	0	1	0	0	0	1	1	0	1	0	1	1	1	0
Received	1	0	1	0	0	0	1	1	0	1	0	1	0	0	1
Error E	0	0	0	0	0	0	0	0	0	0	0	0	1	1	1

E has a 1 exactly where a bit was flipped — here a burst of three

The flipped bits are the last three, so the error polynomial is $E(x) = x^2 + x + 1$ — of **degree 2**.

Example: a short burst error

Our generator is $P(x) = x^5 + x^4 + x^2 + 1$, of **degree 5**.

The **degree** of a polynomial is its highest power — equivalently, the position of its leading 1 bit.

	x^{14}	x^{13}	x^{12}	x^{11}	x^{10}	x^9	x^8	x^7	x^6	x^5	x^4	x^3	x^2	x^1	x^0
Sent T	1	0	1	0	0	0	1	1	0	1	0	1	1	1	0
Received	1	0	1	0	0	0	1	1	0	1	0	1	0	0	1
Error E	0	0	0	0	0	0	0	0	0	0	0	0	1	1	1

E has a 1 exactly where a bit was flipped — here a burst of three

A degree-5 polynomial cannot divide a non-zero one of degree 2, so the remainder cannot be zero: **this error is caught.**

Example: a short burst error

Our generator is $P(x) = x^5 + x^4 + x^2 + 1$, of **degree 5**.

The **degree** of a polynomial is its highest power — equivalently, the position of its leading 1 bit.

	x^{14}	x^{13}	x^{12}	x^{11}	x^{10}	x^9	x^8	x^7	x^6	x^5	x^4	x^3	x^2	x^1	x^0
Sent T	1	0	1	0	0	0	1	1	0	1	0	1	1	1	0
Received	1	0	1	0	0	0	1	1	0	1	0	1	0	0	1
Error E	0	0	0	0	0	0	0	0	0	0	0	0	1	1	1

E has a 1 exactly where a bit was flipped — here a burst of three

This is why we required P 's **highest-order bit to be 1** — it is what holds the degree at r , and the degree is what sets how long a burst can be and still be caught. (The **lowest-order bit** is what makes it hold wherever in the frame the burst falls — the case we skipped here, where the burst is not at the very end.)

Some Standard Polynomials

These trade-offs have already been worked out. **Always use a standard generator — never invent your own.**

Four versions of $P(X)$ are widely used:

- **CRC-12**: transmissions of streams of 6-bit characters
- **CRC-16 / CRC-ANSI**: various communication protocols, including serial standards like Modbus
- **CRC-CCITT**: telecommunications applications
- **CRC-32 / IEEE-802**: network protocols — Ethernet, ZIP, PNG, and other file transfer protocols

CRC-12	$= X^{12} + X^{11} + X^3 + X^2 + X + 1$
CRC-ANSI	$= X^{16} + X^{15} + X^2 + 1$
CRC-CCITT	$= X^{16} + X^{12} + X^5 + 1$
IEEE-802	$= X^{32} + X^{26} + X^{23} + X^{22} + X^{16} + X^{12} + X^{11} + X^{10} + X^8$ $+ X^7 + X^5 + X^4 + X^2 + X + 1$

RECAP

Error detection — what we covered

- An **error-detecting code** appends check bits so the receiver can test whether the data arrived intact.
- A **parity check** adds one bit and misses any even number of flips.
- The **Internet checksum** is cheap to compute and is used by IP, TCP and UDP.
- The **CRC** appends an FCS that makes the frame divide exactly by an agreed generator — the strongest of the three.

MODULE 3 · CONCEPT 3 OF 3

Error Correction

1 · Error Detection

2 · CRC

3 · Error Correction

Forward Error Correction (FEC)

→ Typically, **retransmission** is needed when bit errors are detected.

→ Not good enough for **wireless applications**:

- High **bit error rate (BER)** on a wireless link
- Long **propagation delay** on satellite links

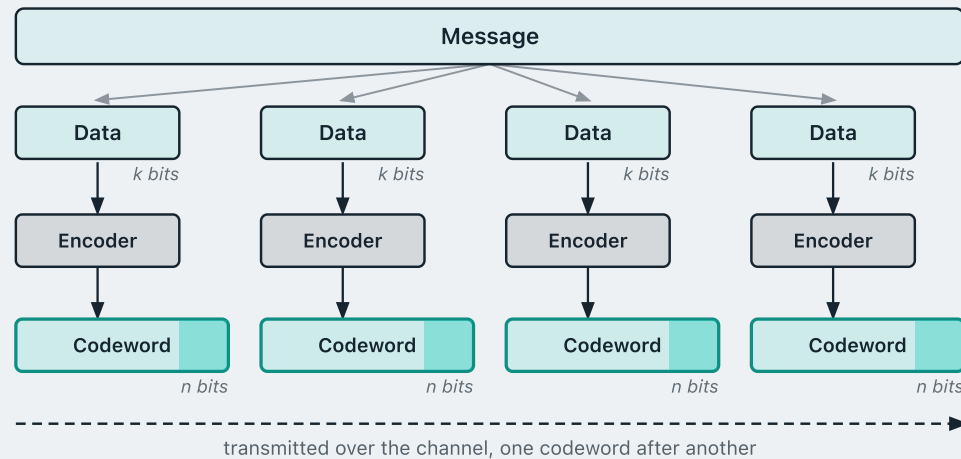
→ Need to **correct errors** on the basis of the bits received — without retransmission.

The FEC process — sender

The message is divided into blocks of k bits — each one a **data block**.

The **encoder** is applied to each data block **independently**, turning every k -bit block into an n -bit **codeword** — the extra $n-k$ bits are the redundancy.

The codewords are then transmitted, one after another.



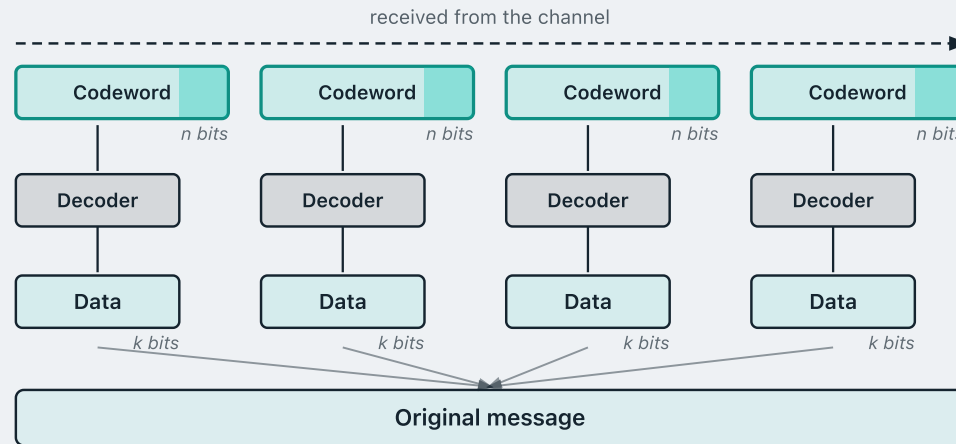
The FEC process — receiver

The receiver splits the incoming stream back into n -bit codewords — some of which may have been corrupted in transit.

The **decoder** is applied to each codeword independently.

Each decode recovers the original k -bit data block, **correcting** any errors the code is strong enough to fix.

The recovered blocks are reassembled into the original message.



Block Code Principles — Hamming Distance

The **Hamming distance** $d(v_1, v_2)$ between two n -bit binary sequences is *the number of bit positions in which v_1 and v_2 disagree*.

Example:

$$\begin{array}{cccccc} v_1 = & 0 & 1 & 1 & 0 & 1 & 1 \\ & \downarrow & \cdot & \downarrow & \cdot & \downarrow & \cdot \\ v_2 = & 1 & 1 & 0 & 0 & 0 & 1 \end{array}$$

3 positions differ $\rightarrow d(v_1, v_2) = 3$

How the encoder and decoder work

Constructing the **encoder** is the most important part — it is what determines how many errors the code can **detect** and **correct**. We will not go into those details here.

You can always think of an encoder as a **map**: a table assigning each k -bit data block one n -bit codeword. That is all we need in this lecture.

But whatever the code, the **decoder** always works in the same way.

for example, with $k = 2$ and $n = 5$:

Data block	Codeword
00	00000
01	00111
10	11001
11	11110

Minimum Hamming distance decoding

Given the received n -bit pattern, the decoder asks:

1. Is it one of the valid codewords?

If so, **no error is detected** — output the data block the encoder assigned to it.

2. If not, an error has occurred — that much we know immediately.

Then find the valid codeword **closest to it** in Hamming distance, and output *that* codeword's data block. This is **minimum-distance decoding**.

Step 2 works because **fewer bit flips are more likely than many** — the nearest valid codeword is the most plausible thing the sender actually sent.

Minimum-distance decoding — an example

Our **encoder**, with $k = 2$ and $n = 5$:

Data block	Codeword
00	00000
01	00111
10	11001
11	11110

Assumption: the channel is lossy, but **at most one bit** per codeword is flipped.

The receiver gets **00100** — which is **not** one of the four valid codewords, so something was corrupted.

Which valid codeword is closest?

Received	Hamming distance	Valid codeword	Data block
00100	1	00000	00
00100	2	00111	01
00100	4	11001	10
00100	3	11110	11

The minimum distance is **1**, to 00000, so the decoder outputs data block **00**. Under our assumption of at most one flip, that **must** be what the transmitter sent — the error is corrected.

When can we actually correct?

Minimum-distance decoding only works if the closest valid codeword is **unique**. It need not be.

The same encoder as before:

Data block	Codeword
00	00000
01	00111
10	11001
11	11110

New assumption: this channel is noisier — **up to two bits** per codeword may be flipped.

The receiver gets **01010**.

Received	Hamming distance	Valid codeword	Data block
01010	2	00000	00
01010	3	00111	01
01010	3	11001	10
01010	2	11110	11

Two codewords sit at distance 2, and two flips are now allowed. We know an error occurred — but 00 and 11 are equally plausible, so the error can be **detected but not corrected**.

The full picture for this code

With $n = 5$ there are $2^5 = 32$ possible received patterns:

00000 00001 00010 ... 00111 ... 11001 ... 11110 11111

Only **four** of them are valid codewords, so $32 - 4 = 28$ are invalid.

- **20 of the 28** are at Hamming distance 1 from a valid codeword.
- Each is at distance 1 from **exactly one** codeword, never two — so its nearest codeword is **unique**.
- Therefore all 20 can be **corrected**. That was our first example.

01010 01011 01100 01101 10010 10011 10100 10101

- The remaining **8** lie at distance 2 from **two different** codewords, so the nearest is **not unique**.
- So they can only be **detected** — our second example.

So this code can **detect** any error of up to two bits, but only **single-bit errors** can be **corrected**.

What determines the ability to correct?

For a **general** code, how many errors can it correct, and how many can it detect?

It comes down to a single number: the **minimum Hamming distance** d of the code — the smallest distance between **any two** valid codewords.

Our code has four valid codewords:

00000

00111

11001

11110

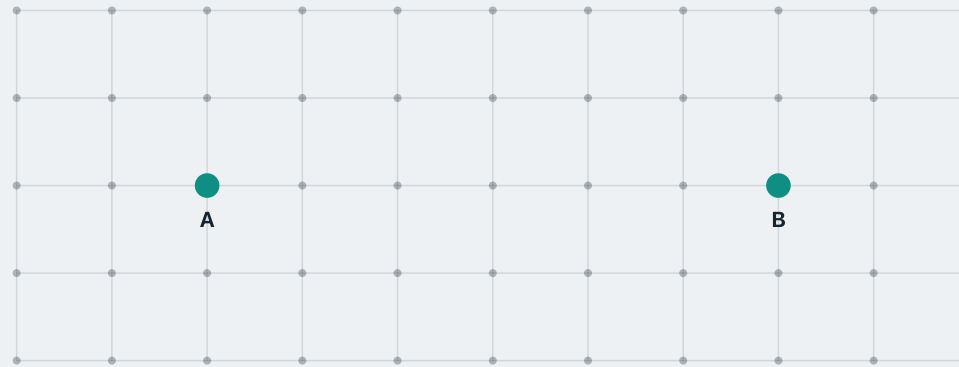
Pair of codewords		Distance
00000	00111	3
00000	11001	3
00000	11110	4
00111	11001	4
00111	11110	3
11001	11110	3

Every pair, six in total.

Reading off the smallest entry: **$d = 3$** .

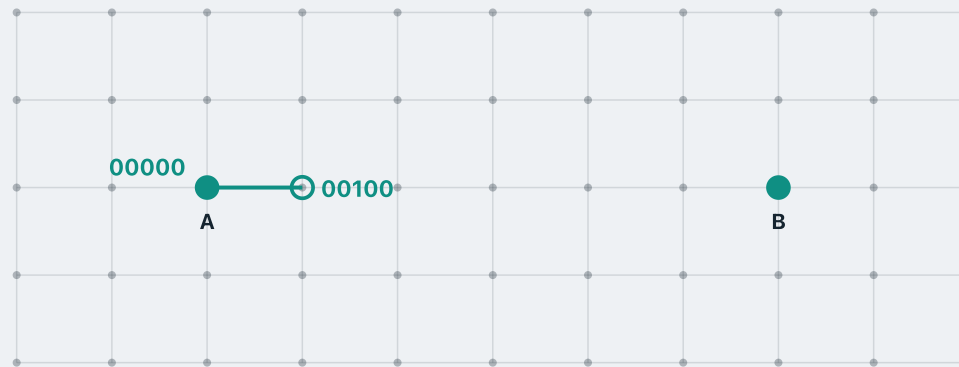
Walking away from a codeword

Picture the whole space of bit strings as a **grid**: every node is one specific bit string, and any two **adjacent** nodes differ in **exactly one bit** — Hamming distance 1.



Walking away from a codeword

For example, if **A** is 00000, then one step away is a string like 00100 — they differ in **exactly one bit**, so their Hamming distance is **1**.



Walking away from a codeword

Step **up** instead and we reach 10000 — a different bit flipped, but still **one step** from 00000.



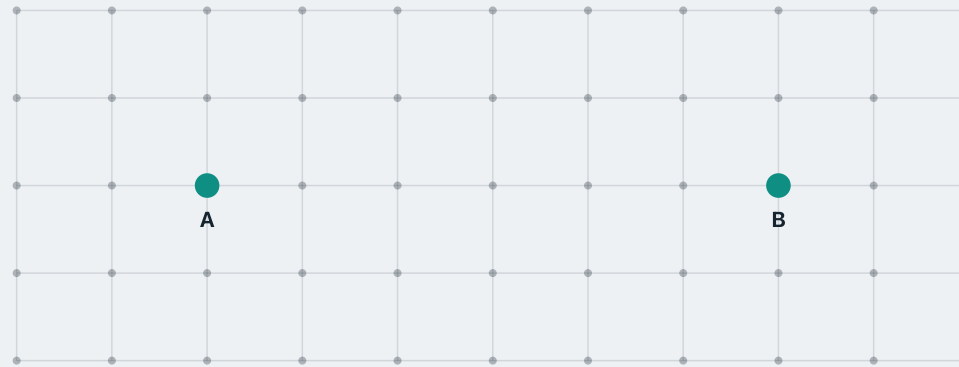
Walking away from a codeword

Notice that 10000 and 00100 are **two steps** apart — two edges to travel from one to the other.



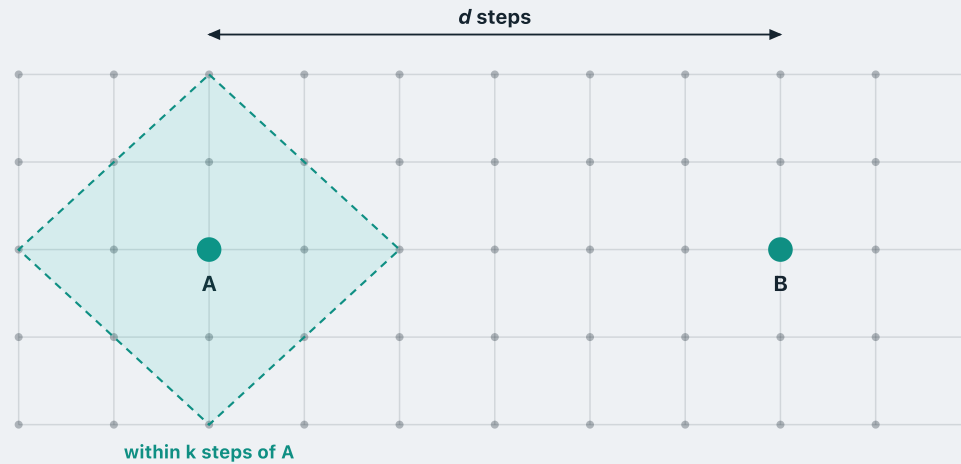
Walking away from a codeword

Now suppose we use a code whose **minimum Hamming distance** is d .



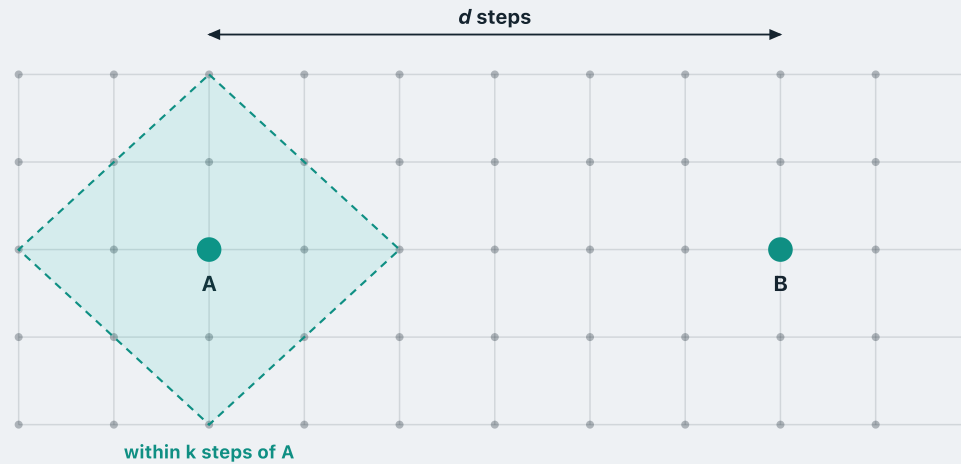
Walking away from a codeword

Send **A** over a channel that flips at most **k** bits. Each flip is one step to a neighbour, so the received pattern lands **somewhere within k steps of A**.



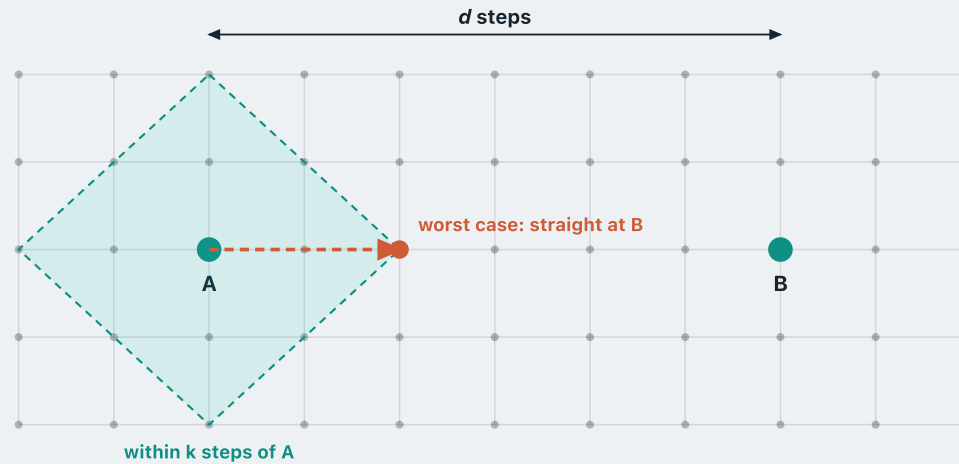
Walking away from a codeword

The further it drifts from A, the **less it looks like A** — and the more it may start to resemble some other codeword.



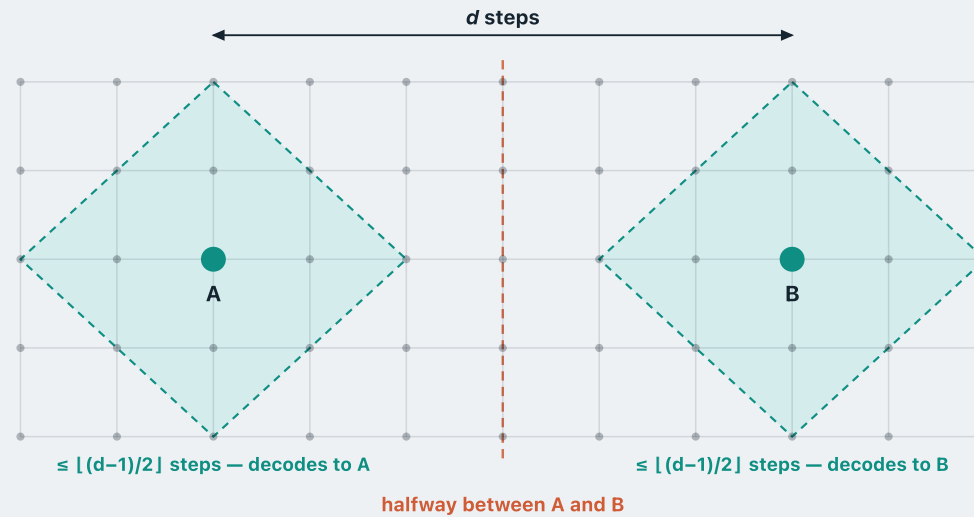
Walking away from a codeword

The **worst case** is when every flip takes us straight towards the **nearest** codeword B, so the pattern looks more and more like B with each step.



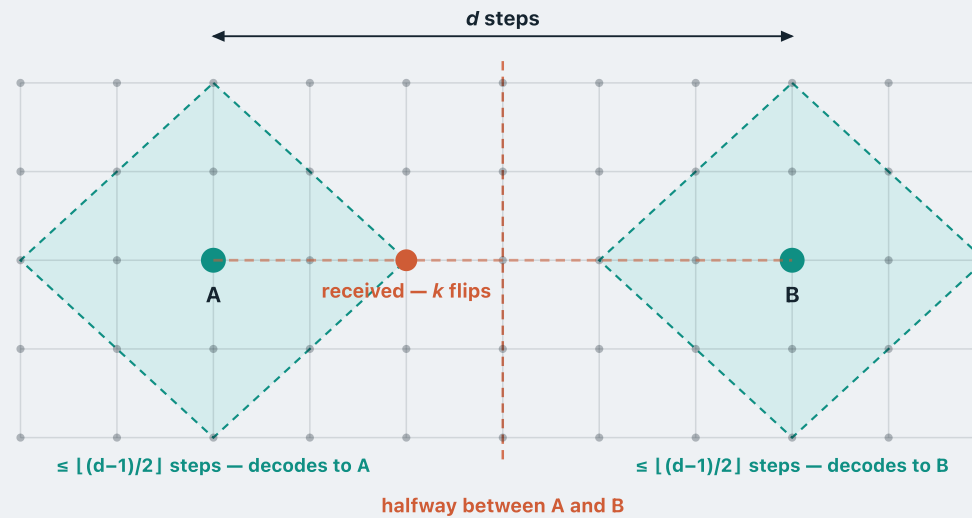
When can the error still be corrected?

Suppose the error flips k bits, with $k \leq \lfloor (d-1)/2 \rfloor$. Then the received pattern stays inside **A's region** and never crosses the **halfway** line — even in the worst case.



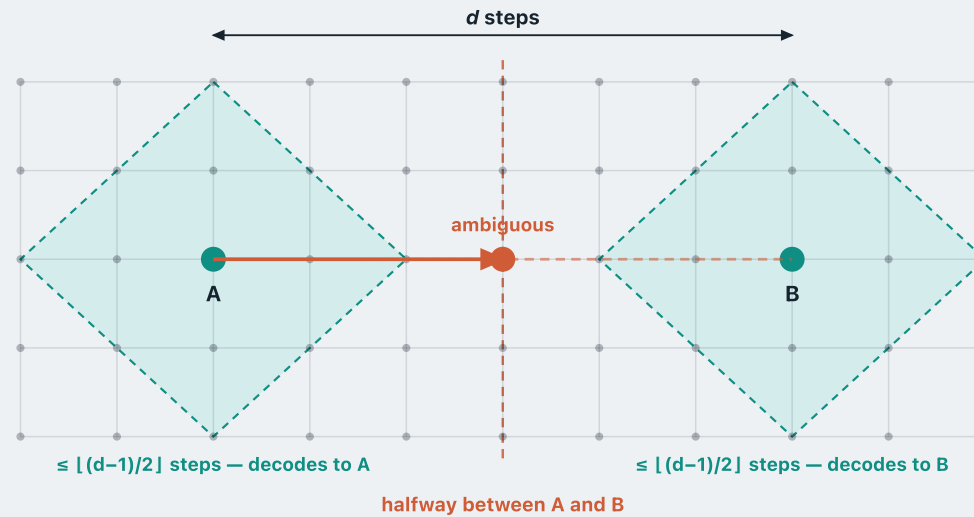
When can the error still be corrected?

So **A** is strictly the nearest codeword, and minimum-distance decoding walks us back to it: a k -bit error is **corrected**.



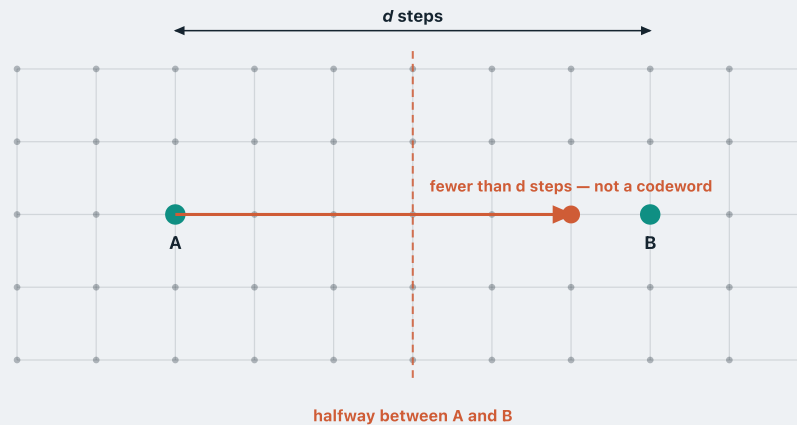
When can the error still be corrected?

Now suppose $k = d/2$. The pattern lands exactly on the **halfway** line, **equidistant** from A and B — it could have come from either, so **correction is no longer possible**.



Past halfway, we can still detect

- Suppose the channel flips **more than $\lfloor (d-1)/2 \rfloor$** bits, so correction is out of reach.
- As long as it flips **fewer than d** bits, we cannot reach B — B is a full d steps away.
- And B is the **nearest** codeword to A, so if we cannot reach B we cannot reach **any** codeword.
- So wherever the pattern lands, it **cannot be a valid codeword**.
- The receiver sees a non-codeword and reports an error — the error is **detected**, even though it cannot be corrected.



The rule

A code with minimum Hamming distance d

- **corrects** up to $\lfloor (d-1)/2 \rfloor$ bit errors
- **detects** up to $d - 1$ bit errors

For the code we have been working with, $d = 3$ — so it corrects **1** error and detects **2**.

So a good FEC code is designed to make its **minimum distance as large as possible** — the larger d is, the more errors it can correct and detect.

RECAP

Error correction — what we covered

Forward error correction repairs errors at the receiver itself, from the bits that arrived — **without retransmission**.

The **encoder** turns every k -bit data block into an n -bit **codeword**, with $n > k$; the extra $n-k$ bits are the redundancy.

The **decoder** checks whether the received pattern is a valid codeword; if it is not, an error is **detected**, and **minimum-distance decoding** returns the data block of the nearest codeword.

How many errors it can detect and correct hinges entirely on the code's **minimum Hamming distance** d .

References

James F. Kurose & Keith W. Ross, *Computer Networking: a top-down approach featuring the Internet*, Addison Wesley, 3rd edition, 2005.

- Chapter 5 — The Link Layer and Local Area Network (§5.2)

William Stallings, *Data and Computer Communications*, Pearson, 10th edition, 2013.

- Chapter 6 — Error Detection and Correction

Andrew S. Tanenbaum & David J. Wetherall, *Computer Networks*, Prentice Hall, 5th edition, 2010.

- Chapter 3 — The Data Link Layer (§3.2)

Acknowledgements

These lecture notes are developed based on slides from the following sources:

- **Dr Barry Wu**'s slides for COSC264, University of Canterbury, 2024–25
- **Dr Mengmeng Ge**'s lecture notes for COSC264, University of Canterbury, 2023
- **Assoc Prof Dan Kim**'s lecture notes for COSC264, University of Canterbury, 2017
- **Prof Aleksandar Kuzmanovic**'s lecture notes for CS340, Northwestern University, 2005

Lineage & colophon

These lecture notes modernise COSC264 material developed by **Dr Barry Wu** (2024–25), building on notes by **Dr Mengmeng Ge** (2023) and **Assoc Prof Dan Kim** (2017), University of Canterbury, and lecture notes by **Prof Aleksandar Kuzmanovic** (CS340, Northwestern University, 2005).